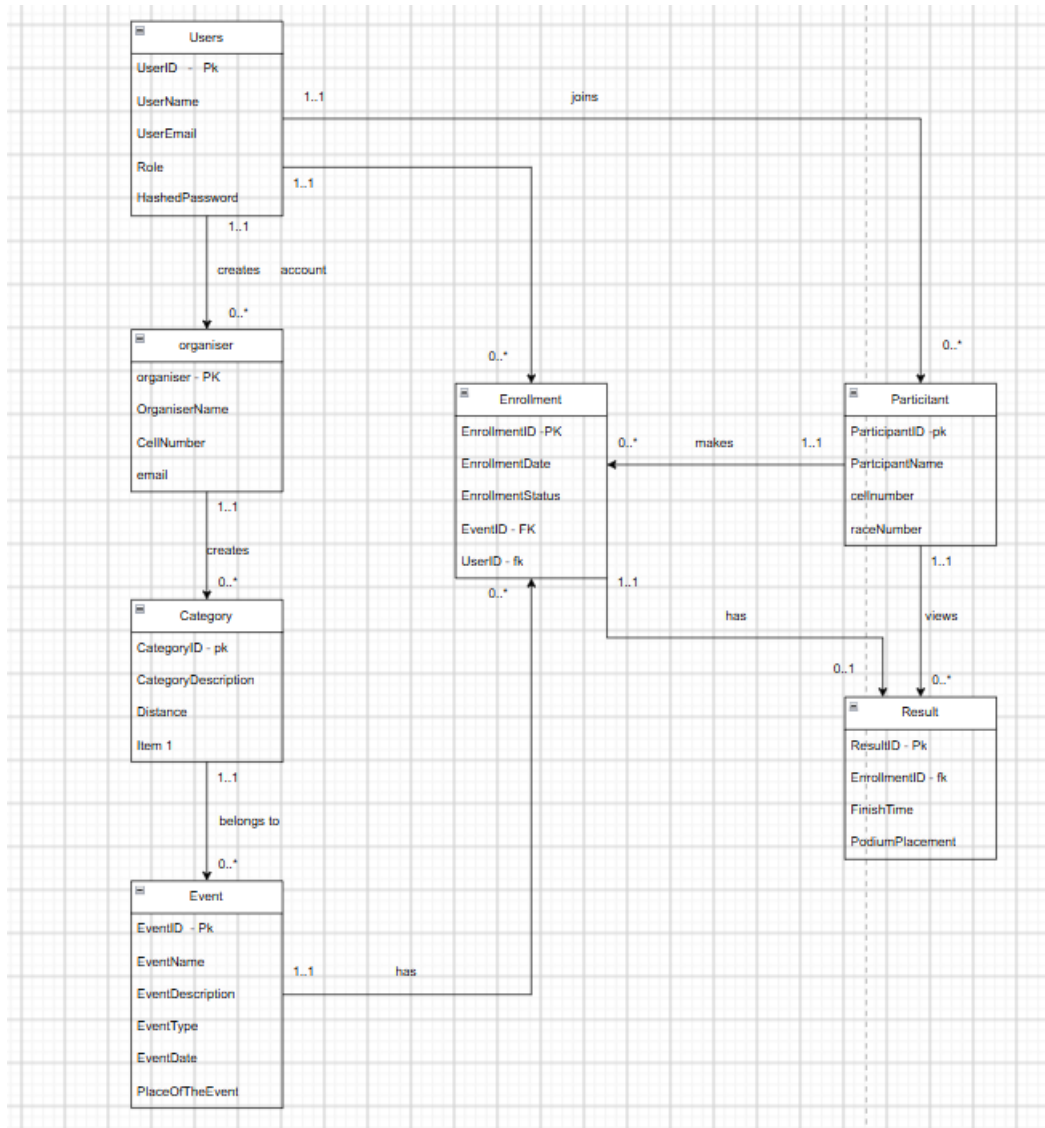


Section A

The ERD



(draw.io, 2026)

Business rules

1. A User can create an account and log in to the system.
2. A User can either be an Organiser or a Participant, depending on their role.
3. An Organiser can create many Categories for different race types or age groups.
4. Each Category belongs to one Event, and one Event can have many Categories.
5. An Organiser can create many Events, but each Event is created by one Organiser.

6. A Participant can join many Events, but each Enrollment record links one Participant to one Event.
7. Each Enrollment keeps track of when a participant joined and their status (confirmed, pending, or cancelled).
8. Each Enrollment can have one Result, showing the participant's finish time and podium placement.
9. A Participant can view their Results after the event is done. A participant can view multiple results, and many results can be viewed by one participant
10. The system must store all User, Event, Category, Enrollment, and Result data for history and tracking.

Section B

API endpoint table

Http Methods	Route	Description	Role Required	Request Body	Expected Response
Post	/api/auth/register	For registering a new user(organiser or participant)	public	{name, email, password, role}	201 created, user registered successfully
Post	/api/auth/login	Authenticates user and returns JWT token	public	{email, password}	200 ok, token returned if not 401 unsuccessful
Get	/api/catergory	View all categories	Organiser/participant	none	200 OK, list of events.
Post	/api/category	Creates a new category	Organiser	{categoryDescription, distance, eventId}	201 created, added category
Get	/aip/event	View all events	Organiser/participant	none	200 ok, list of events
Post	/api/event	Create a new event	Organiser	{eventName, eventDescription, eventType, eventDate, placeOfEvent}	201 created-events
Put	/api/event/{id}	Update event details	Organiser	{eventName, eventDescription, eventDate, placeOfEvent}	200 OK, event update unless 404 no found
Delete	/api/event/{id}	Delete an event	Organiser	none	204 no content/404 not found
Post	/api/enrolment/	Enroll participant into an event	participant	{eventId, userId}	201 Created – Enrollment recorded.
Post	/api/result	Records race results for a participant	Organiser	{enrollmentId, finishTime, podiumPlacement}	201 created, result saved
Get	/api/result/{participantId}	View results for a participant	participant	results	200 Ok, list of result

Section C

Database Part

--Crating a database

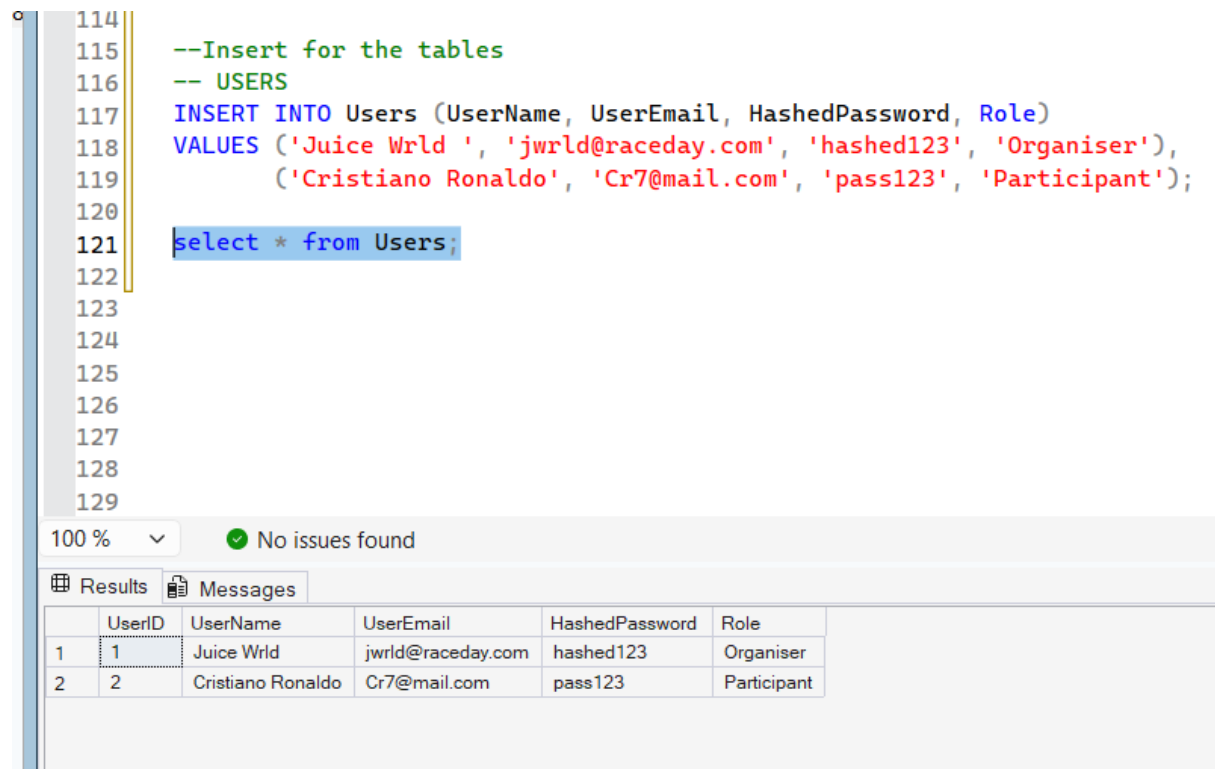
```
create database Prog;
```

```
use Prog;
```

-- creating users table

```
CREATE TABLE Users (  
    UserID INT PRIMARY KEY IDENTITY(1,1),  
    UserName VARCHAR(100) NOT NULL,  
    UserEmail VARCHAR(100) UNIQUE NOT NULL,  
    HashedPassword VARCHAR(255) NOT NULL,  
    Role VARCHAR(20) NOT NULL -- role = Organiser or Participan  
);
```

Output



The screenshot shows a SQL IDE interface. The top pane displays SQL code for creating a table and inserting data. The bottom pane shows the results of the execution, including a table with two rows and a status bar indicating 'No issues found'.

```
114  
115 --Insert for the tables  
116 -- USERS  
117 INSERT INTO Users (UserName, UserEmail, HashedPassword, Role)  
118 VALUES ('Juice Wrld ', 'jwrlld@raceday.com', 'hashed123', 'Organiser'),  
119          ('Cristiano Ronaldo', 'Cr7@mail.com', 'pass123', 'Participant');  
120  
121 select * from Users;  
122  
123  
124  
125  
126  
127  
128  
129
```

100 % No issues found

Results Messages

	UserID	UserName	UserEmail	HashedPassword	Role
1	1	Juice Wrld	jwrlld@raceday.com	hashed123	Organiser
2	2	Cristiano Ronaldo	Cr7@mail.com	pass123	Participant

-- organiser table

```
CREATE TABLE Organiser (  
    OrganiserID INT PRIMARY KEY IDENTITY(1,1),  
    OrganiserName VARCHAR(100) NOT NULL,  
    CellNumber VARCHAR(20),  
    Email VARCHAR(100),  
    UserID INT NOT NULL,  
    FOREIGN KEY (UserID) REFERENCES Users(UserID)  
);
```

```
123  -- Organiser  
124  INSERT INTO Organiser (OrganiserName, CellNumber, Email, UserID)  
125  VALUES ('Juice Wrld', '0821112222', 'jwrld@raceday.com', 1);  
126  
127  
128  select * from Organiser;  
129  
130  
131
```

100 % No issues found

Results Messages

	OrganiserID	OrganiserName	CellNumber	Email	UserID
1	1	Juice Wrld	0821112222	jwrld@raceday.com	1

-- category table

```
CREATE TABLE Category (  
    CategoryID INT PRIMARY KEY IDENTITY(1,1),  
    CategoryDescription VARCHAR(100) NOT NULL,  
    Distance VARCHAR(50),  
    Item1 VARCHAR(50),  
    OrganiserID INT NOT NULL,  
    FOREIGN KEY (OrganiserID) REFERENCES Organiser(OrganiserID)  
);
```

```

131  -- Categyory
132  INSERT INTO Category (CategoryDescription, Distance, Item1, OrganiserID)
133  VALUES ('10km Run', '10km', 'Under 20', 1);
134
135  select * from Category;

```

100 % No issues found

Results Messages

	CategoryID	CategoryDescription	Distance	Item1	OrganiserID
1	1	10km Run	10km	Under 20	1

-- event table

CREATE TABLE Event (

EventID INT PRIMARY KEY IDENTITY(1,1),

EventName VARCHAR(100) NOT NULL,

EventDescription VARCHAR(255),

EventType VARCHAR(50),

EventDate DATE NOT NULL,

PlaceOfTheEvent VARCHAR(100),

CategoryID INT NOT NULL,

FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)

);

```

140
141 -- EVENT
142 INSERT INTO Event (EventName, EventDescription, EventType, EventDate, PlaceOfTheEvent, CategoryID)
143 VALUES ('Joburg Marathon', 'Annual city marathon', 'Run', '2026-10-15', 'Johannesburg', 1);
144
145 Select * from Event;
146
147
148
149
150
151
152
153

```

100 % No issues found

Results Messages

	EventID	EventName	EventDescription	EventType	EventDate	PlaceOfTheEvent	CategoryID
1	1	Joburg Marathon	Annual city marathon	Run	2026-10-15	Johannesburg	1

-- participant table

CREATE TABLE Participant (

ParticipantID INT PRIMARY KEY IDENTITY(1,1),

ParticipantName VARCHAR(100) NOT NULL,

CellNumber VARCHAR(20),

Email VARCHAR(100),

RaceNumber VARCHAR(20),

UserID INT NOT NULL,

FOREIGN KEY (UserID) REFERENCES Users(UserID)

);

```

140
149 -- Participant
150 INSERT INTO Participant (ParticipantName, CellNumber, Email, RaceNumber, UserID)
151 VALUES ('Thabo Nkosi', '0833334444', 'thabo@mail.com', 'R001', 2);
152
153 select * from Participant;
154
155
156
157
158

```

100 % No issues found

Results Messages

	ParticipantID	ParticipantName	CellNumber	Email	RaceNumber	UserID
1	1	Thabo Nkosi	0833334444	thabo@mail.com	R001	2

-- enrollment tables

```
CREATE TABLE Enrollment (  
    EnrollmentID INT PRIMARY KEY IDENTITY(1,1),  
    EnrollmentDate DATE NOT NULL,  
    EnrollmentStatus VARCHAR(50) NOT NULL,  
    UserID INT NOT NULL,  
    EventID INT NOT NULL,  
    FOREIGN KEY (UserID) REFERENCES Users(UserID),  
    FOREIGN KEY (EventID) REFERENCES Event(EventID)  
);
```

```
-- enrollment  
insert into Enrollment (EnrollmentDate, EnrollmentStatus, UserID, EventID)  
VALUES ('2026-09-01', 'Confirmed', 2, 1);  
  
select * from Enrollment;
```

-- RESULT TABLE

```
CREATE TABLE Result (  
    ResultID INT PRIMARY KEY IDENTITY(1,1),  
    EnrollmentID INT NOT NULL,  
    FinishTime TIME,  
    PodiumPlacement INT,  
    FOREIGN KEY (EnrollmentID) REFERENCES Enrollment(EnrollmentID)  
);
```

```
164
165 -- result insert
166 INSERT INTO Result (EnrollmentID, FinishTime, PodiumPlacement)
167 VALUES (1, '03:45:10', 1);
168
169
170 select * from Result;
171
172
```

100 % No issues found

Results Messages

	ResultID	EnrollmentID	FinishTime	PodiumPlacement
1	1	1	03:45:10.0000000	1